

Pascal Lexer with Flex

This project contains a simple lexical analyzer (lexer) for the Pascal programming language, implemented using Flex (the fast lexical analyzer generator).

Files

- **pascal.l**: The Flex specification file containing rules for recognizing Pascal tokens.
- **sample.pas**: A sample Pascal source file to test the lexer.

How It Works

The lexer reads Pascal source code and prints out the type of each token it recognizes. Each rule in `pascal.l` matches a specific pattern in the input and prints a message indicating the token type.

Rule Explanations

- **Keywords**: Matches Pascal reserved words (e.g., `program`, `var`, `begin`, `end`, etc.).
 - Example: `"program" { printf("KEYWORD: program\n"); }`
- **Identifiers**: Matches variable and procedure names (start with a letter, followed by letters or digits).
 - Pattern: `[a-zA-Z][a-zA-Z0-9]*`
 - Action: Prints `IDENTIFIER: <name>`
- **Numbers**:
 - **Real numbers**: `[0-9]+\.[0-9]+` (e.g., `3.14`)
 - **Integers**: `[0-9]+` (e.g., `42`)
- **Operators and Delimiters**: Matches Pascal symbols like `:=`, `;`, `,`, `(`, `)`, `:`, `=`, `<=`, `>=`, `<>`, `<`, `>`, `.`, `+`, `-`, `*`, `/`.
 - Each has its own rule and prints a corresponding token name (e.g., `SEMICOLON`, `ASSIGN`, `DOT`, etc.).
- **Whitespace**: `[\t\n\r]+` skips spaces, tabs, newlines, and carriage returns (no output).
- **Unknown Characters**: `.` matches any character not handled above and prints `UNKNOWN: <char>`.

Example Output

Given the following Pascal code in `sample.pas`:

```
program Example;
var x, y: integer;
begin
  x := 10;
  y := x + 20;
end.
```

The lexer will output:

```
KEYWORD: program
IDENTIFIER: Example
SEMICOLON
KEYWORD: var
IDENTIFIER: x
COMMA
IDENTIFIER: y
COLON
KEYWORD: integer
SEMICOLON
KEYWORD: begin
IDENTIFIER: x
ASSIGN
INTEGER: 10
SEMICOLON
IDENTIFIER: y
ASSIGN
IDENTIFIER: x
PLUS
INTEGER: 20
SEMICOLON
KEYWORD: end
DOT
```

How to Build and Run

Prerequisites

- **Flex** and **gcc** must be installed. On Ubuntu/WSL:

```
sudo apt update  
sudo apt install flex gcc
```

Steps

1. **Generate the C source from the Flex file:**

```
flex pascal.l
```

2. **Compile the generated C code:**

```
gcc lex.yy.c -lfl -o pascal_lexer
```

3. **Run the lexer on a Pascal file:**

```
./pascal_lexer < sample.pas
```